

Performance Modeling of a Key-Value-Stationary Systolic Array for Multi-Query Attention Inference

Project Proposal

Hudson Strauss

Dhruv Patel

May 23, 2026

High-Performance Computer Architecture and Systems
Santa Clara University

1 Introduction

Transformer inference has made memory-efficient attention a central systems problem. During autoregressive decoding, each generated token repeatedly accesses the key-value (KV) cache, causing memory traffic to scale with sequence length. As context windows increase, inference becomes increasingly memory-bound rather than compute-bound.

Multi-Query Attention (MQA) reduces this overhead by sharing keys and values across multiple query heads, substantially lowering KV-cache footprint compared to standard multi-head attention (Ainslie et al., 2023). However, most accelerators still map attention onto generic matrix multiplication hardware despite the streaming nature of autoregressive decoding.

Traditional systolic-array dataflows such as output-stationary, weight-stationary, and input-stationary mappings are optimized for dense GEMM workloads. They do not directly exploit the fact that during MQA inference, queries stream continuously while keys and values remain relatively fixed.

This project investigates whether a specialized KV-stationary systolic-array mapping is a better architectural match for MQA decoding. The central hypothesis is that keeping KV data resident near compute while streaming queries through the array will reduce memory movement, improve processing-element (PE) utilization, and lower latency per generated token.

Figure 3 illustrates the proposed KV-stationary dataflow used throughout the performance model.

2 Problem Statement

The goal of this project is to evaluate a KV-stationary systolic-array dataflow for MQA inference and compare it against conventional systolic mappings.

For each incoming query vector Q , attention decoding proceeds in three stages:

$$s_i = Q \cdot K_i$$

$$a_i = \frac{e^{s_i}}{\sum_j e^{s_j}}$$

$$O = \sum_i a_i V_i$$

where:

Q	Incoming query vector
K_i	Key stored in PE i
V_i	Value vector stored in PE i
s_i	Local attention score
a_i	Normalized attention weight

The proposed architecture maps this flow directly onto a systolic fabric by keeping keys and values resident within the array while query vectors stream through the compute pipeline.

3 Proposed Architecture

Figure 3 shows the proposed KV-stationary MQA architecture.

Each processing element stores a resident key-value pair (K_i, V_i) . Incoming query vectors stream horizontally across the array and interact with stationary KV data.

Each PE performs three operations:

1. Local score computation

$$s_i = Q \cdot K_i$$

2. Online softmax update

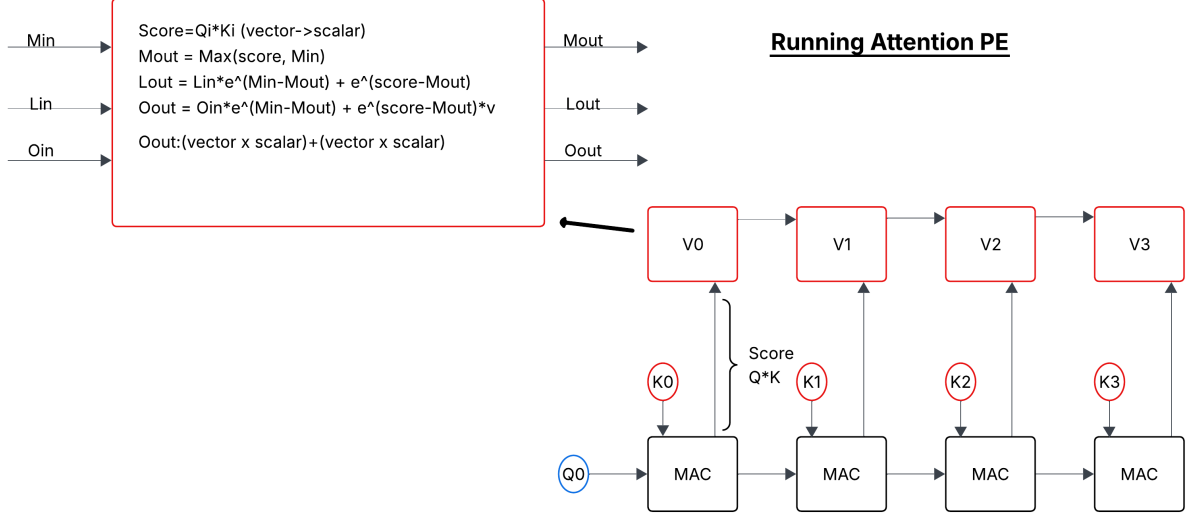
$$m_{\text{out}} = \max(m_{\text{in}}, s_i)$$

$$l_{\text{out}} = l_{\text{in}} e^{(m_{\text{in}} - m_{\text{out}})} + e^{(s_i - m_{\text{out}})}$$

3. Weighted value accumulation

$$O_{\text{out}} = O_{\text{in}} e^{(m_{\text{in}} - m_{\text{out}})} + V_i e^{(s_i - m_{\text{out}})}$$

The online softmax formulation avoids materializing the full attention matrix and enables pipelined normalization directly within the systolic dataflow.



Each processing element stores a resident key-value pair (K_i, V_i) , while query vectors stream horizontally across the array.

Each PE computes a local score $s_i = Q \cdot K_i$, updates online softmax state (m, l, O) , and forwards partial outputs to the next PE.

Final normalization produces the output vector:

$$\text{Output} = \frac{O_N}{l_N}$$

This architecture avoids materializing the full attention score vector and reduces repeated KV-cache reads during autoregressive decoding.

Each processing element stores a resident key-value pair (K_i, V_i) , while query vectors stream horizontally across the array.

Each PE computes a local score $s_i = Q \cdot K_i$, updates online softmax state (m, l, O) , and forwards partial outputs to the next PE.

Final normalization produces the output vector:

$$\text{Output} = \frac{O_N}{l_N}$$

This architecture avoids materializing the full attention score vector and reduces repeated KV-cache reads during autoregressive decoding.

Figure 1: Proposed KV-stationary MQA dataflow.

Each processing element stores a resident key-value pair (K_i, V_i) , while query vectors stream horizontally across the array.

Each PE computes a local score $s_i = Q \cdot K_i$, updates online softmax state (m, l, O) , and forwards partial outputs to the next PE.

Final normalization produces the output vector:

$$\text{Output} = \frac{O_N}{l_N}$$

This architecture avoids materializing the full attention score vector and reduces repeated KV-cache reads during autoregressive decoding.

4 Methodology and Evaluation

The proposed KV-stationary mapping will be compared against three baseline attention implementations:

Materialized attention baseline
FlashAttention-style block streaming baseline
Conventional systolic GEMM mappings

All designs will be evaluated under identical workload and hardware constraints to isolate the impact of the dataflow itself.

The experiments will sweep:

Sequence length	128 to 1024 tokens
Head dimension	32, 64, 128
Batch size	1
Array dimensions	16×16 to 64×64
Memory model	Bandwidth-constrained
Precision	INT8

The evaluation focuses on edge-style autoregressive inference where latency and memory movement dominate performance.

Primary evaluation metrics include:

Total execution cycles	Latency per generated token	Throughput
KV-cache traffic reduction	PE utilization	On-chip storage requirements

The project will analyze the workload conditions under which KV-stationary execution outperforms conventional systolic mappings.

5 Risks and Limitations

One technical challenge is accurately modeling online softmax and reduction behavior within systolic-array simulations, since most simulators naturally optimize for GEMM-style computation.

To address this limitation, matrix-dominant operations will be modeled directly while softmax-related reduction stages will be approximated using a consistent analytical latency model.

Another risk is that KV-stationary execution may only provide advantages under long-context workloads where memory traffic dominates total latency. Experiments will therefore include both short and long sequence lengths to identify crossover points rather than relying on a single favorable regime.

6 Expected Outcomes

The expected outcome is a comparative performance analysis demonstrating whether KV-stationary systolic execution is a better architectural match for MQA decoding than conventional systolic-array mappings.

The project aims to determine:

When KV-stationary execution improves latency
How much KV-cache traffic can be reduced
Whether online softmax fusion improves pipeline efficiency
Which workload regimes benefit most from the proposed mapping

7 References

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebrón, F., & Sanghai, S. (2023). Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. <https://aclanthology.org/2023.emnlp-main.298/>